

파이썬 비동기 프로그래밍 기초

동시 처리 극대화

동시 처리 극대화

여러 비동기 작업을 동시에 처리하려면 어떤 방식을 써야할까?

비효율성 (순차 실행)

작업 A

작업 B

작업 C

작업 A

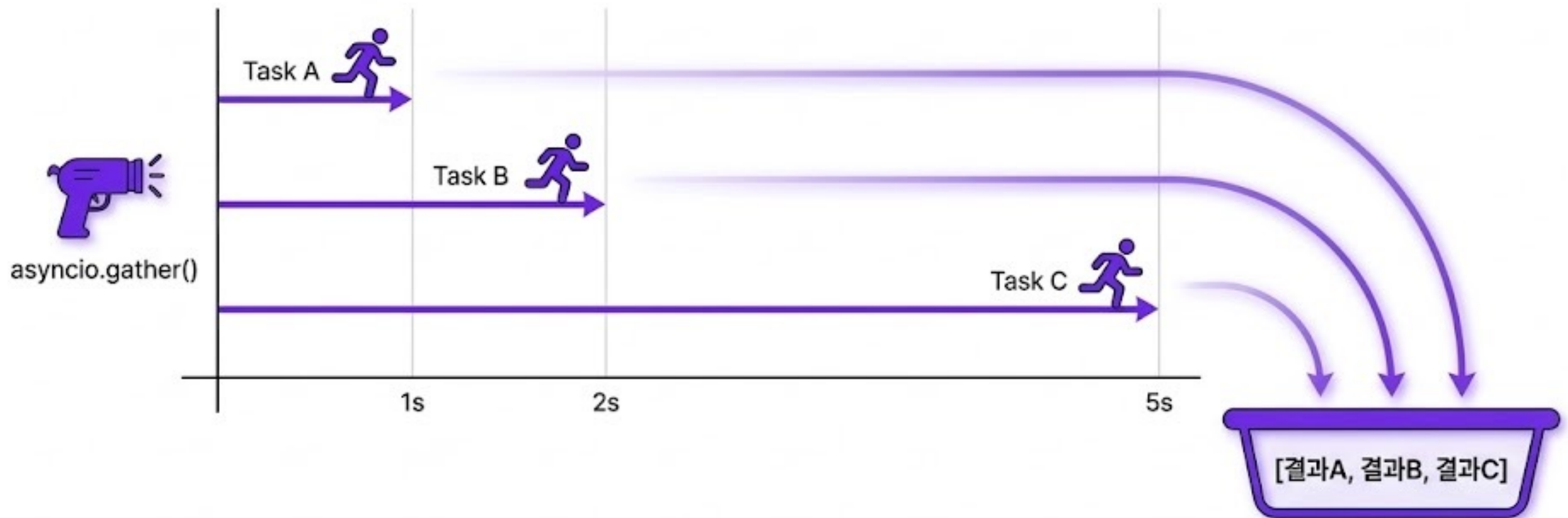
작업 B

작업 C

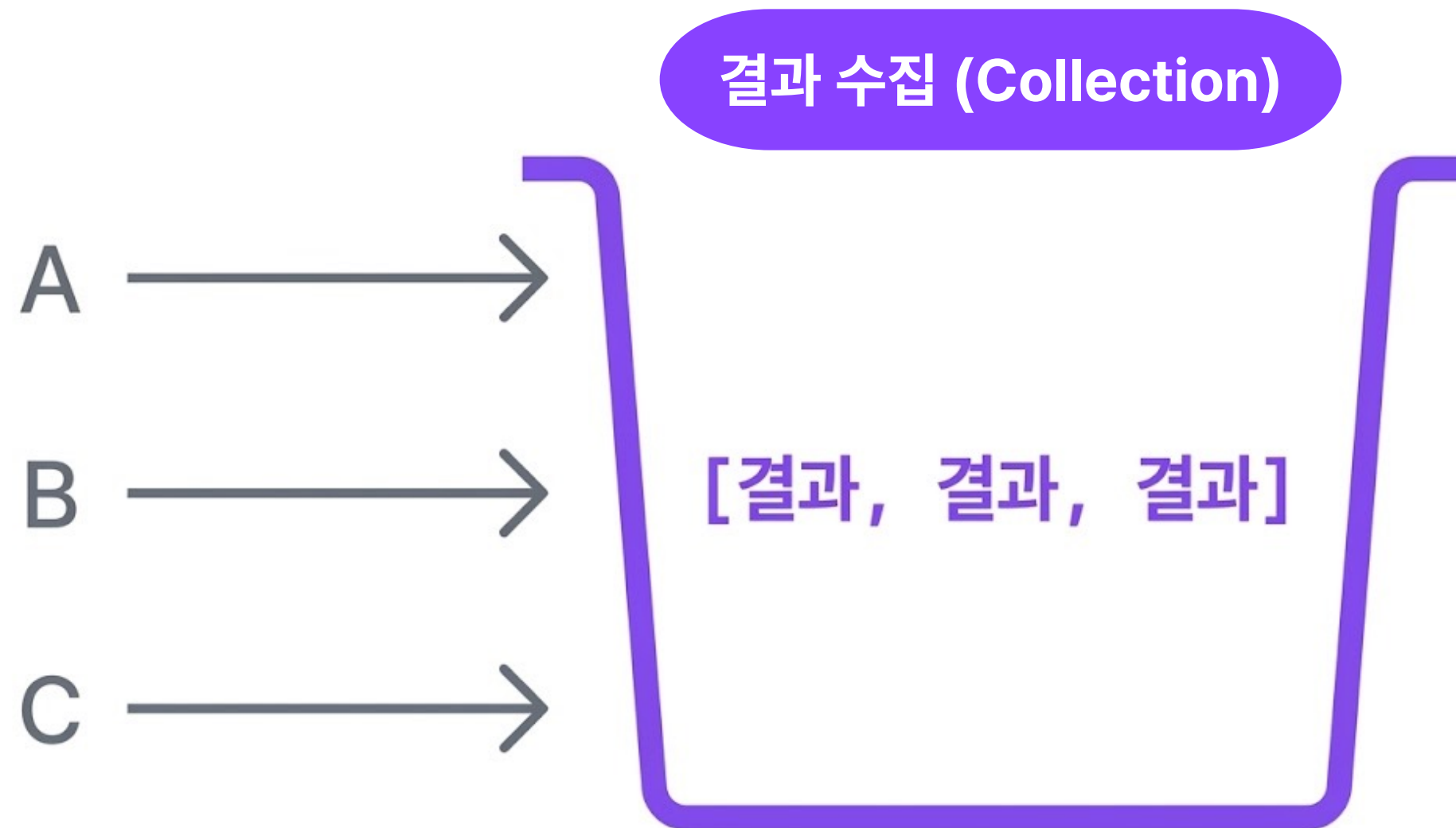
동시 처리의 극대화

여러 개의 독립적인 비동기 작업을 한 번에 묶어 실행
전체 실행 시간은 가장 오래 걸리는 단일 작업의 시간과 동일

1초 + 2초 + 5초 $\neq$ 8초
총 소요 시간 = 5초 (Max)



첫 번째 접근 : 결과를 한 번에 모으는 수집가



- `asyncio.gather()`는 여러 코루틴을 한 번에 실행함
- 모든 작업이 끝나면 그 결과들을 리스트(List) 형태로 모아서 반환
- 입문 단계에서 가장 직관적이고 널리 쓰이는 패턴



단일 대기에서 동시 실행으로의 코드 전환

순차 실행

```
start = time.perf_counter()
```

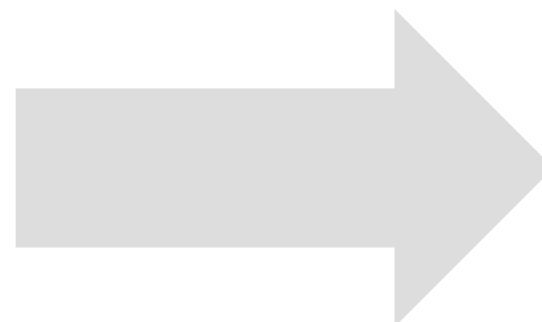
```
await task('A', 1)
```

```
await task('B', 2)
```

```
await task('C', 3)
```

소요 시간: 약 6초

개별 await를 하나의
gather()로 묶어냄



동시 실행

```
start = time.perf_counter()
```

```
await asyncio.gather(
```

```
    task('A', 1),
```

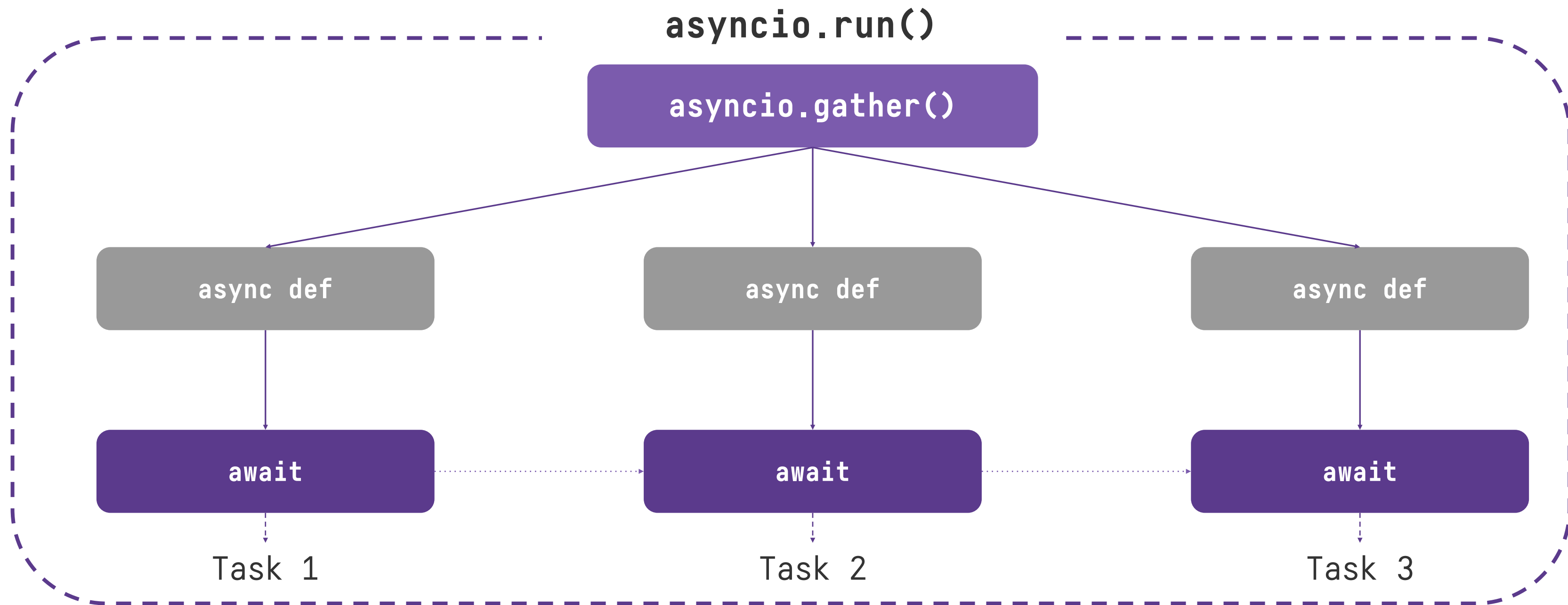
```
    task('B', 2),
```

```
    task('C', 3)
```

```
)
```

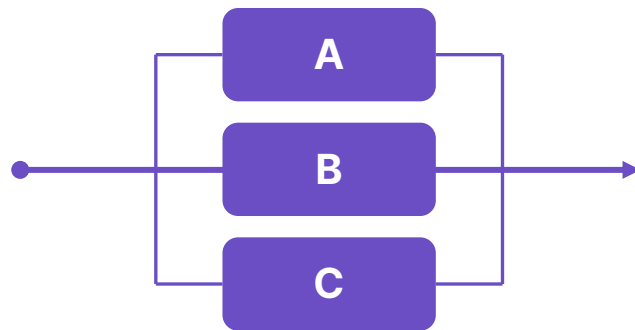
소요 시간: 약 3초

비동기 생태계의 완성



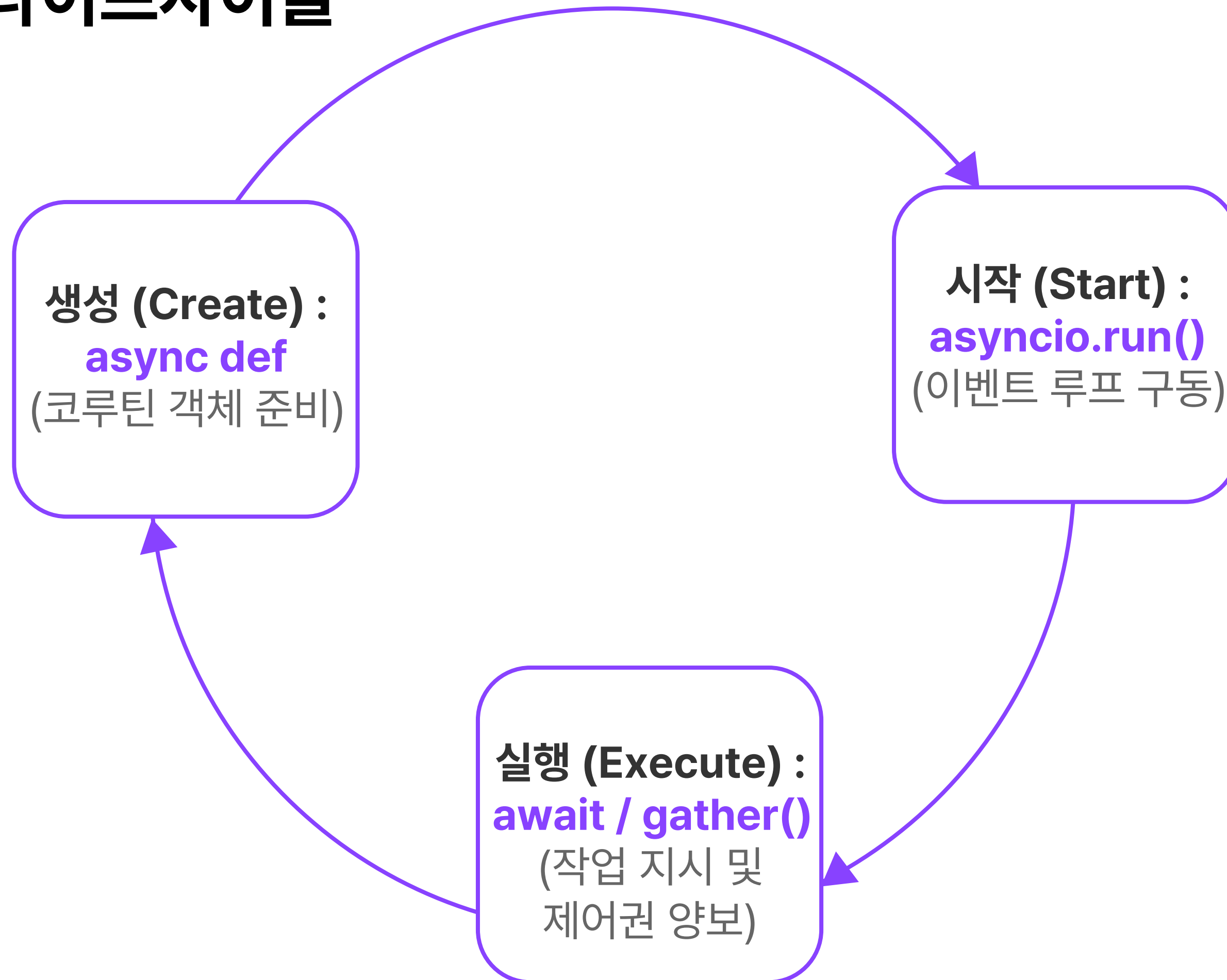
실행 흐름 전체의 구조를 순차적(Sequential)에서 **협력적(Cooperative)**로 재설계

상황에 맞는 최적의 실행 전략 선택하기

	순차 실행 (Sequential)	동시 실행(asyncio.gather)
실행 타임라인		
총 실행 시간	모든 작업 시간의 합 (A+B+C)	가장 긴 작업 시간과 유사 (Max)
이상적인 사용처	이전 작업의 결과가 다음 작업에 필요한 경우 (의존성 O)	5번의 독립적인 API 요청과 같이 순서가 무관한 경우 (의존성 X)
주요 리스크	병목 현상으로 인한 비효율적인 시간 낭비	한 번에 과도한 요청으로 인한 시스템 과부하



파이썬 비동기 라이프사이클





Gather 결과

worker A, B, C를

모아 실행하면,

반환값은 하나의 리스트

results에 담깁니다.

```
import asyncio

async def worker(name: str, delay: int):
    await asyncio.sleep(delay)
    return f"{name} done"

async def main():
    results = await asyncio.gather(
        worker("A", 1),
        worker("B", 2),
        worker("C", 1),
    )
    print(results)

asyncio.run(main())
```

한 번에 모아 기다리기 : `asyncio.gather()`

```
import asyncio

async def fetch(url):
    await asyncio.sleep(1)
    return f"{url} done"

async def main():
    results = await asyncio.gather(
        fetch("api/1"),
        fetch("api/2"),
        fetch("api/3")
    )
    print(results)

# ['api/1 done', 'api/2 done', 'api/3 done']
```

```
await asyncio.gather(*코루틴들)
```

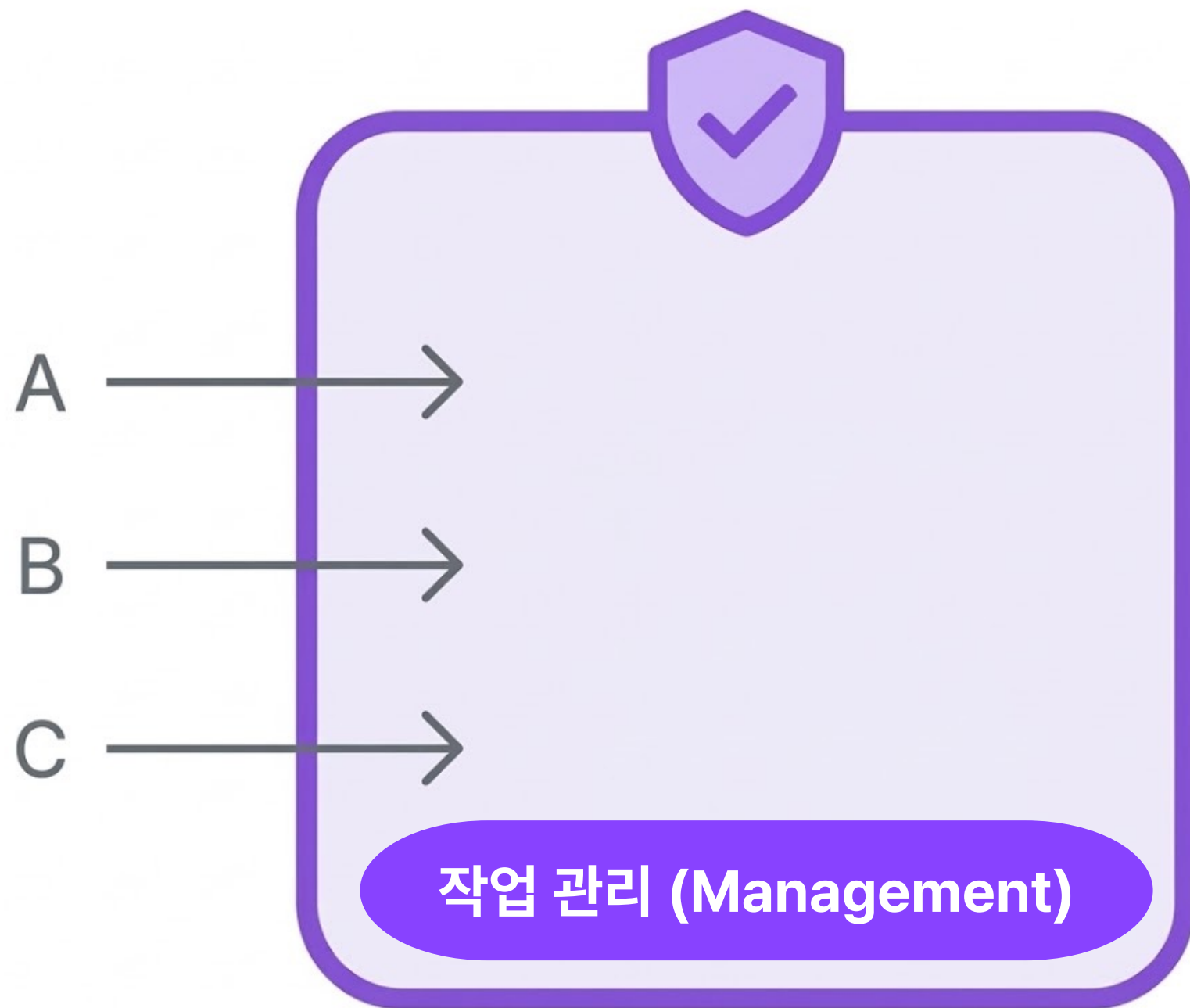
인자: 여러 **awaitable** (콤마로 나열)

반환값: 결과 **리스트** (입력 순서 보장)

핵심 포인트

- 모든 코루틴을 **동시에 실행**하고 전부 끝나면 결과 수집
- 결과 리스트 **순서 보장** (입력 순서 = 출력 순서)

두 번째 접근 : 안전한 울타리를 제공하는 관리자



- Python 3.11 이상에서 도입된 `asyncio.TaskGroup()`
- 관련된 작업들을 하나의 그룹(`Context()`)으로 묶어 더 안전하게 생명주기를 관리
- 모든 작업이 완료될 때까지 기다리고, 예외 발생 시 모든 작업을 안전하게 취소



비동기 반복과 컨텍스트 : `async for` & `async with`

```
# async for
async for item in aiter:
    print(item)
```

`async for`

- 비동기 이터레이터를 **순회**
- 내부적으로 각 반복마다 `await`로 다음 값을 가져옴
- 예: 스트리밍 응답, DB 커서

```
# async with
async with aiohttp.ClientSession() as s:
    resp = await s.get(url)
```

`async with`

- 비동기 **컨텍스트 매니저**
- 블록 진입 시 **연결 열기**, 종료 시 **자동 정리**
- 예: HTTP 세션, DB 연결, 파일

`async def` 안에서만 사용 가능 | 일반 `for` / `with`와 구조는 동일



with문 (컨텍스트 매니저)

```
# with 없이 직접 관리
f = open("data.txt")
data = f.read()
f.close() # 깜빡하면 자원 누수!

# with문으로 자동 관리
with open("data.txt") as f:
    data = f.read()
# 블록을 벗어나면 자동으로 close()
```

with 표현식 as 변수:
...

진입 시 `__enter__()` 호출 → 자원 준비

종료 시 `__exit__()` 호출 → 자동 정리

핵심 포인트

- 코드 블록을 **컨텍스트**로 감싸는 구문
- 예외가 발생해도 **정리 코드 실행 보장**
- 파일, DB 연결, 네트워크 등 **자원 관리**에 사용



TaskGroup 결과

Context Manager(**async with**)를 사용하여 **tg**라는 울타리를 만듭니다.

이 블록을 빠져나갈 때 모든 작업의 완료(또는 안전한 취소)가 보장됩니다.

```
import asyncio

async def worker(name: str, delay: int):
    await asyncio.sleep(delay)
    print(f"{name} done")

async def main():
    async with asyncio.TaskGroup() as tg:
        tg.create_task(worker("A", 1))
        tg.create_task(worker("B", 2))
        tg.create_task(worker("C", 1))

asyncio.run(main())
```



안전한 울타리 : `asyncio.TaskGroup`

```
import asyncio

async def fetch(url):
    await asyncio.sleep(1)
    return f"{url} done"

async def main():
    async with asyncio.TaskGroup() as tg:
        t1 = tg.create_task(fetch("api/1"))
        t2 = tg.create_task(fetch("api/2"))
    print(t1.result(), t2.result())

# 블록 끝 = 모든 태스크 완료 보장
```

```
async with asyncio.TaskGroup() as tg:
    task = tg.create_task(코루틴)
```

클래스: **비동기 컨텍스트 매니저**

메서드: **`tg.create_task()`** → Task 객체 반환

조건: **Python 3.11+**

핵심 포인트

- with 블록이 끝나면 **모든 태스크 완료 보장**
- 하나가 실패하면 **나머지도 자동 취소** (안전)
- 결과는 **`task.result()`**로 개별 조회



TaskGroup의 결과 조회와 예외 처리

```
# 결과 조회: 블록 종료 후
async with asyncio.TaskGroup() as tg:
    t1 = tg.create_task(fetch("api/1"))
    t2 = tg.create_task(fetch("api/2"))
# 블록 끝 = 모든 태스크 완료 보장
print(t1.result(), t2.result())
```

```
# 예외 처리: ExceptionGroup
try:
    async with asyncio.TaskGroup() as tg:
        tg.create_task(may_fail())
except* ValueError as eg:
    for e in eg.exceptions:
        print(e)
```

결과 조회 방법

- **task.result()**로 개별 조회
- with 블록이 끝나면 **모든 태스크 완료 보장**
- gather와 달리 **리스트가 아닌 개별 Task**

예외 처리 방식

- 하나가 실패 → **나머지 자동 취소**
- 모든 예외를 **ExceptionGroup**으로 묶어서 발생
- **except*** 문법으로 타입별 분리 처리 (3.11+)



겉보기엔 비슷하지만, 성격이 다름

Gather()는 결과 수집 

TaskGroup은 안전한 묶음 관리 



비교 기준	<code>asyncio.gather()</code>	<code>asyncio.TaskGroup()</code>
주요 목적	결과를 한 번에 리스트로 받기	연관된 작업을 하나의 그룹으로 묶기
실패 전파(에러)	하나가 실패해도 나머지는 계속 진행됨	하나가 실패하면 그룹 내 나머지 작업도 즉시 안전하게 취소됨
관리의 특성	각 작업이 독립적일 때 유리	작업들 간의 의존성이 높고 구조적 안정성이 필요할 때 유리



실무 적용 가이드: 언제 무엇을 써야 할까?



입문 & 기본 패턴

gather()

단순한 병렬 실행이 필요하거나, 여러 API 호출의 '결과값들을 합치는 것'이 유일한 목적일 때 가장 직관적이고 편함



실무 & 확장 포인트

TaskGroup

예외 처리가 중요해지고, 하나의 작업 실패가 다른 작업에 영향을 미치는 실무 환경에서 더 튼튼하고 구조적인 코드를 만듦



한 눈에 보는 비동기 실행 파이프라인

Step 1. 선언

async def

(비동기 함수 정의)

>

Step 2. 대기/실행

await

(코루틴의 결과를 기다리며 제어권 양보)

>

Step 3. 동시 조율

asyncio.gather()

(여러 비동기 작업을 한 번에 묶어서 동시 실행)

>

Step 4. 진입점

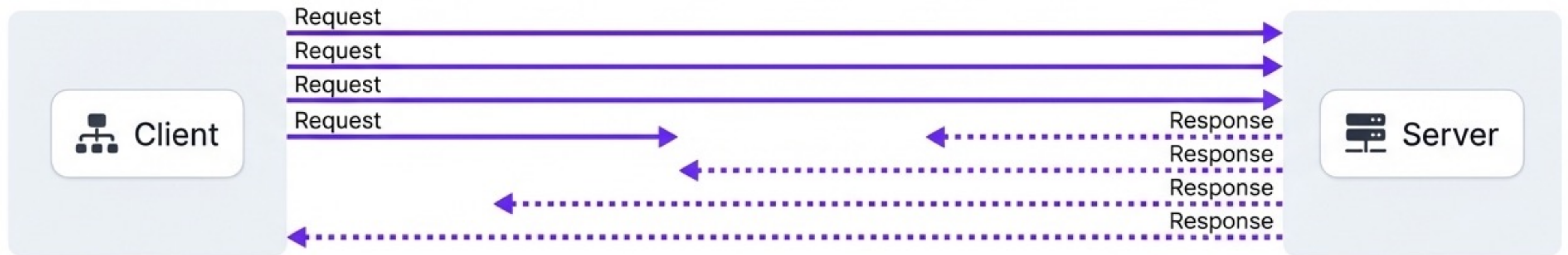
asyncio.run()

(동기 세계에서 비동기 세계로 진입하는 메인 입구)

각각의 키워드는 독립적으로 작동하지 않으며, 이 파이프라인을 따라 유기적으로 연결될 때 비로소 완성됨

실전 비동기 아키텍처

대규모 네트워크 API 요청 시, 응답 순서와 무관하게 준비된 데이터부터 즉각적으로 처리하여 애플리케이션의 반응성 극대화



```
> Task 1: 200 OK (0.8s)
> Task 4: 200 OK (1.1s)
> Task 2: 200 OK (1.5s)...
```



실전 응용 : 여러 API 동시 호출

```
import asyncio
import time
import httpx

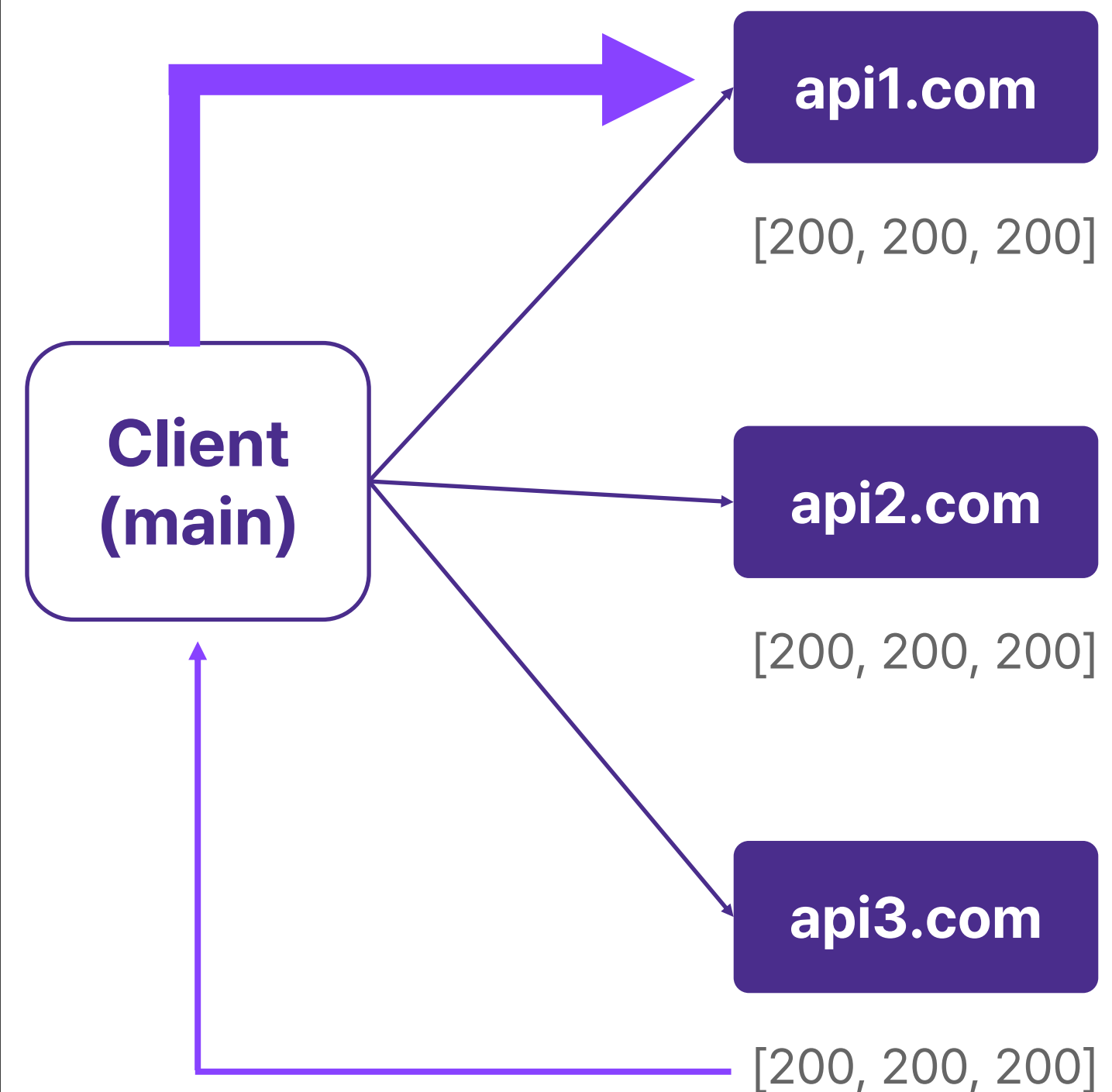
URLS = [
    "https://www.api1.com",
    "https://www.api2.com",
    "https://www.api3.com",
]

async def fetch_url(url):
    client = httpx.AsyncClient()
    response = await client.get(url)
    await client.aclose()
    return {"url": url, "status": response.status_code}

async def fetch_all(urls):
    tasks = [fetch_url(url) for url in urls]
    results = await asyncio.gather(*tasks)
    return list(results)

async def main():
    start = time.time()
    results = await fetch_all(URLS)

asyncio.run(main())
```





httpx.AsyncClient 문법 상세

```
import httpx

# 방법 1: 수동 관리
client = httpx.AsyncClient()
resp = await client.get(url)
await client.aclose() # 반드시 닫기!

# 방법 2: async with (권장)
async with httpx.AsyncClient() as client:
    resp = await client.get(url)
# 블록 종료 시 자동 aclose()
```

httpx.AsyncClient()

→ 비동기 HTTP 클라이언트 **생성**

await client.get(url)

→ 비동기 GET 요청, **Response** 반환

await client.aclose()

→ 연결 **명시적 종료** (누락 시 경고)

핵심 포인트

- **async with** 사용 시 자동 정리 (권장)
- 연결 풀 재사용 → **다중 요청에 효율적**
- .post() .put() .delete() 등 **HTTP 메서드 지원**



목적에 맞는 문법 선택

목적	문법	핵심 특징
기본 실행	<code>asyncio.run</code>	모든 비동기 작업의 최초 진입점
대기/양보	<code>await</code>	순차적 실행 및 제어권 반환
동시 실행	<code>asyncio.gather</code>	병렬 처리 (가장 긴 시간에 동기화)
백그라운드	<code>create_task</code>	대기 없이 즉시 실행 예약